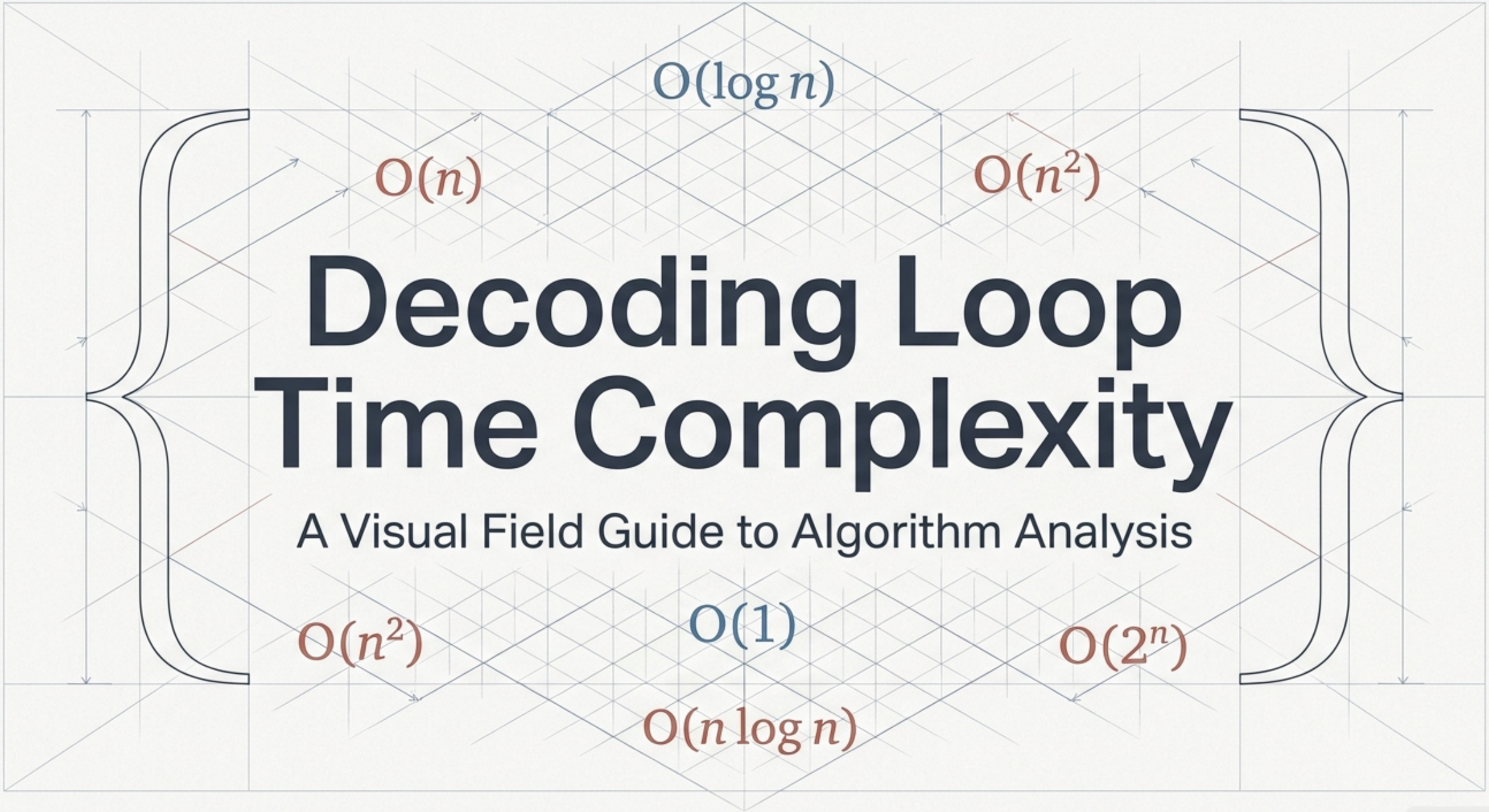
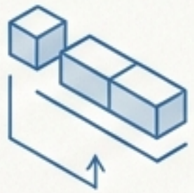
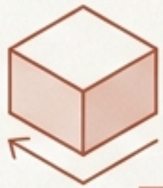


Gemini Notebook

The Universal Equation of Loop Analysis

$T \longrightarrow$ **Total Time Complexity** $\overset{\Delta}{=} \sum$ **[Worst-Case Iterations]** $\times$ $T(n)$ **[Time Per Iteration]**





- Final Big-O Estimation
- N (Problem Size) $\longrightarrow T(n)$
- Constant Operations / Inner Block

The Inspection Dashboard

1. **Start** (Initial Value) — 0
2. **End** (Final Value) — N
3. **Condition** (Limit) — < N, ≤ N, != N
4. **Step** (Increment/Decrement) — +1, -1, += i

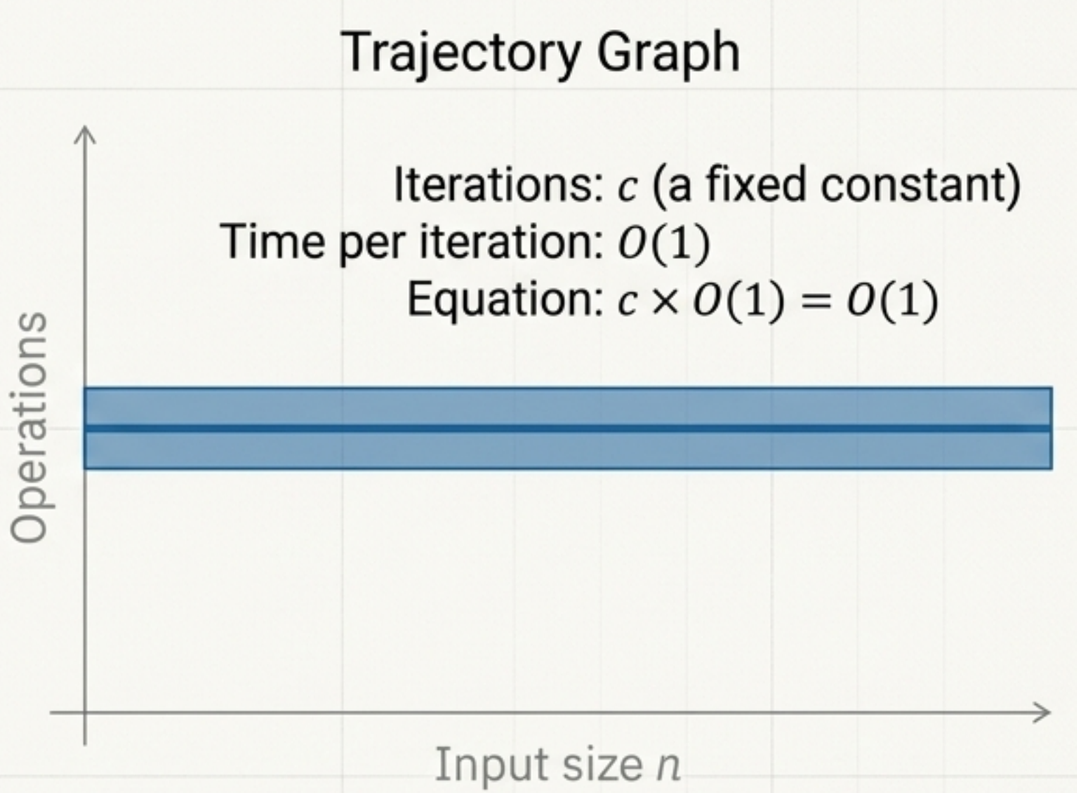
Warning: Lower-order terms and coefficients are ignored in final Big-O notation.

Gemini Notebook

Constant Time: The Unscaling Baseline

```
for(int i = 1; i <= c; i = i + 1)
{
    Some O(1) expressions
}

// while loop version
int i = 1;
while(i <= c)
{
    Some O(1) expressions
    i = i + 1;
}
```



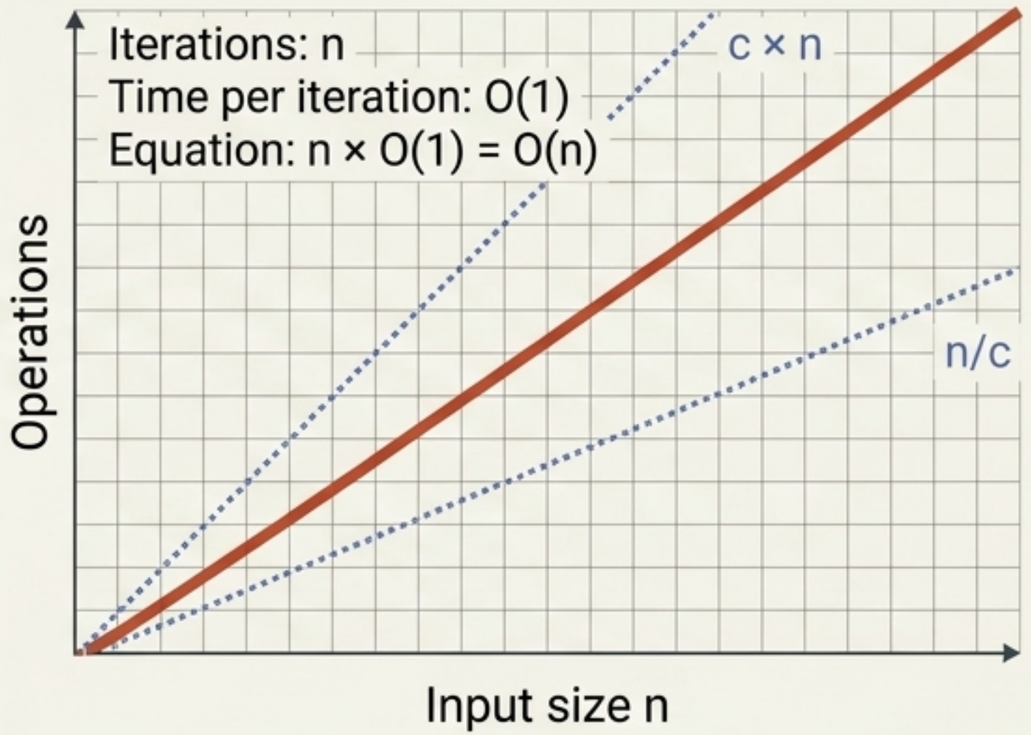
If the loop's execution count does not depend on an external input n, the time remains a flat constant block, no matter how large the constant.

Gemini Notebook

Linear Time: 1-to-1 Scaling

```
for(int i = 1; i <= n; i = i + 1)
{
    Some O(1) expressions
}
```

Trajectory Graph



Whether the loop runs exactly n, n/c, or c × n times, the structural growth remains linear. The Big-O notation absorbs the constant coefficient.

Gemini Notebook

Linear Time via Two-Pointers

```
int l = 0;
int r = n - 1;
while(l <= r)
{
    // Process elements at l and r
    Some O(1) expressions;

    // Move pointers
    if (condition_l) {
        l = l + 1;
    } else if (condition_r) {
        r = r - 1;
    }
}
```

The Pincer Array

Condition: `l <= r`
Mechanic: Increment `l` OR decrement `r`



L (l=0)

R (r=n-1)

Because the variables start at opposite ends and stop when they cross, they collectively traverse the distance of n exactly once. Time complexity is strictly bound to O(n).

Gemini Notebook

Logarithmic Time: Multiplicative Scaling

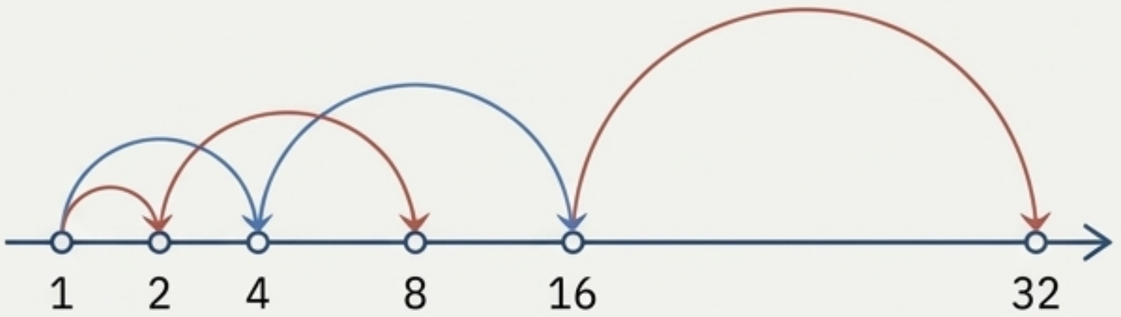
```
for(int i = 1; i < n; i = i * 2)
{
    Some O(1) expressions
}
```

```
for(int i = n; i > 0; i = i / 2)
{
    Some O(1) expressions
}
```

Logarithmic Number Line

Mechanic: Variable increases/decreases by a factor of 2.

Equation: $2^k = n \Rightarrow k = \log_2 n$



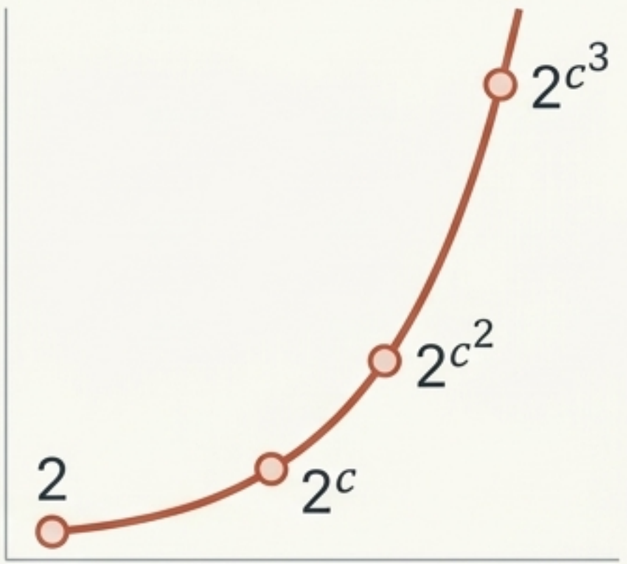
By halving the remaining distance or doubling the step size, it takes incredibly few iterations to reach massive numbers. O(log n) is the signature of extreme efficiency.

Gemini Notebook

Double Logarithmic Time: Exponential Power Jumps

```
//
for(int i = 2; i <= n; i = i^c)
```

The Rocketship Trajectory



Termination:

$$2^{c^k} \leq n$$

Steps taken (k):

$$\approx \log_c \log_2 n$$

2 → 4 → 16 → 256 → 65,536 (in just 5 steps)

The fastest-growing variable results in the fastest-shrinking time complexity. $O(\log \log n)$ is a near-instantaneous bound for almost any practical input.

Gemini Notebook

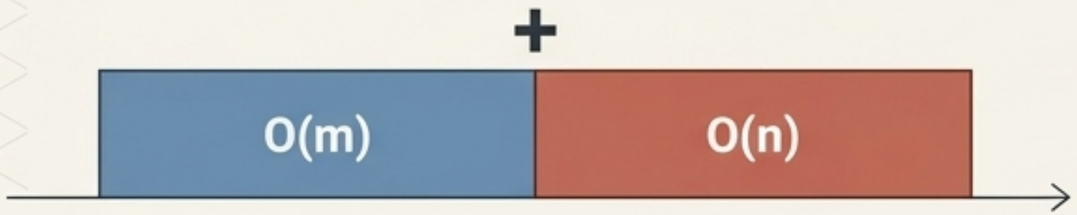
Architecture Rule 1: Consecutive Loops

```
// First independent loop
for(int i = 0; i < m; i++) {
    // Some O(1) operations
}

// Second independent loop
for(int j = 0; j < n; j++) {
    // Some O(1) operations
}
```

The Timeline Blocks

- **Execution:** Sequential
- **Operation:** Addition
- **Equation:** O(m) + O(n) = O(m + n)



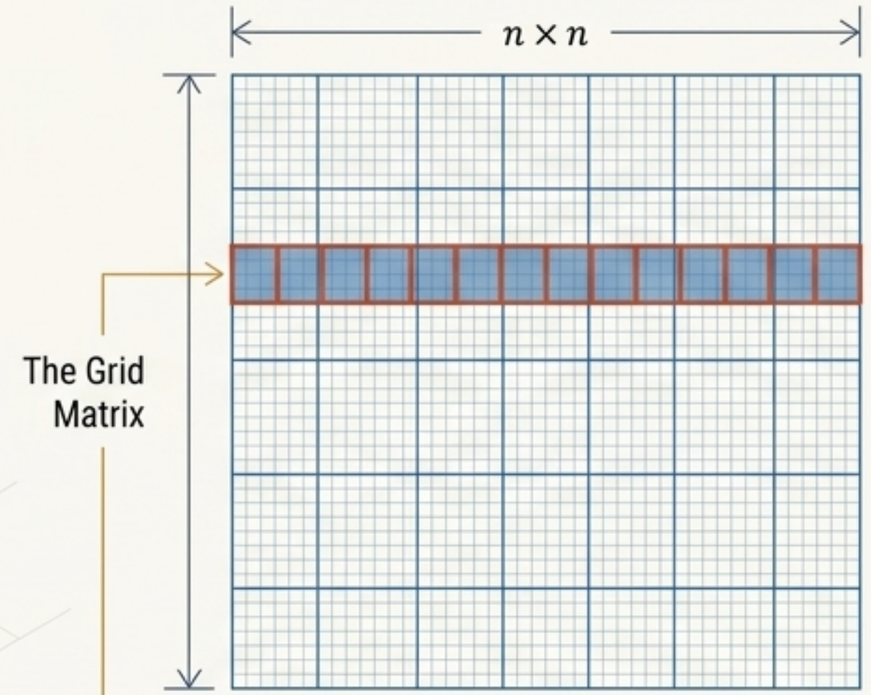
When loops run one after the other entirely independently, their complexities are simply summed together.

Gemini Notebook

Architecture Rule 2: Nested Loops (2D Depth)

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        // Some O(1) operations
    }
}
```

Outer loop iterations: n
Inner loop iterations per outer step: n
Equation: $n \times n \times O(1) = O(n^2)$



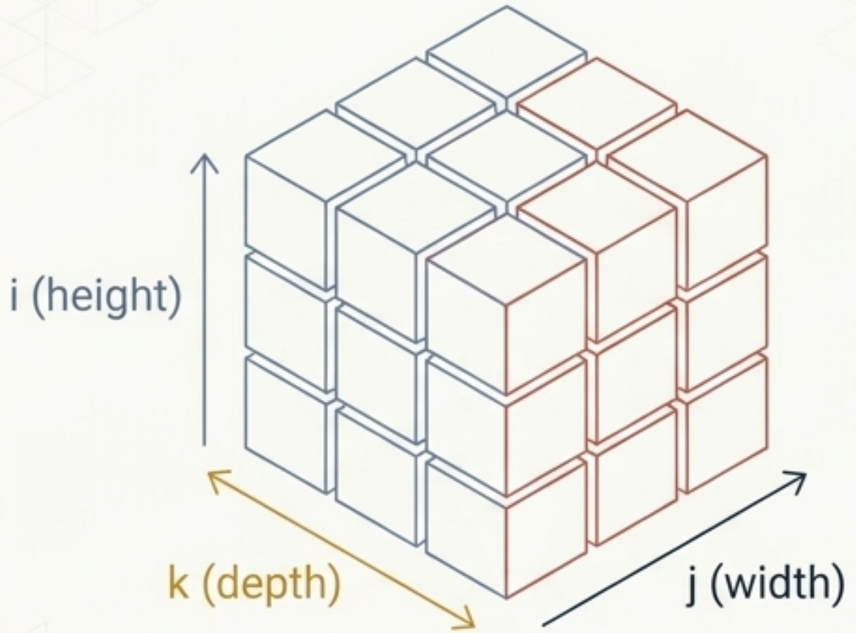
Embedded execution is multiplication. The inner loop must complete its entire cycle for *every single tick* of the outer loop, resulting in a 2D area of operations.

Gemini Notebook

Expanding Depth: 3D Nesting

```
for (int i = 0; i < n; i++) {
  for (int j = 0; j < n; j++) {
    for (int k = 0; k < n; k++) {
      // Some O(1) operations
    }
  }
}
```

- Execution — Three levels deep
- Equation — $n \times n \times n \times O(1) = O(n^3)$



Adding architectural depth exponentially increases operations. A triple-nested loop processes a cubic volume of data.

Gemini Notebook

Hybrid Combinations & Dominant Terms

```
// Single loop O(n)
for(int i = 0; i < n; i++)
{
    // Some O(1) operations
}

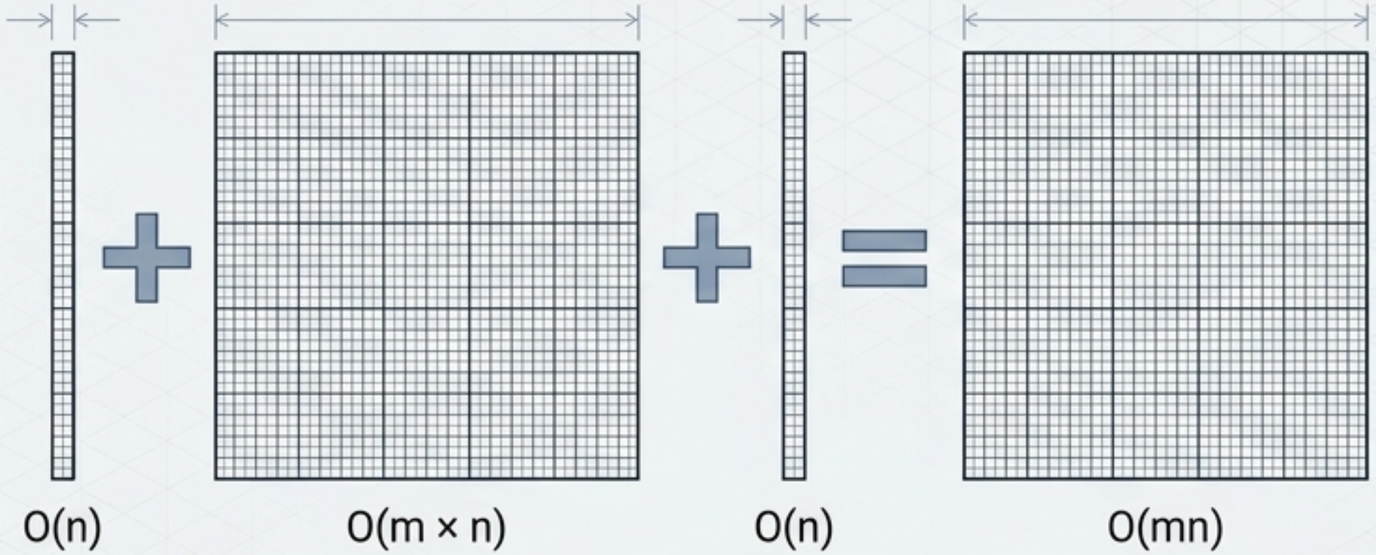
// Nested loops O(mn)
for(int i = 0; i < m; i++)
{
    for(int j = 0; j < n; j++)
    {
        // Some O(1) operations
    }
}

// Single loop O(n)
for(int i = 0; i < n; i++)
{
    // Some O(1) operations
}
```

Visual Equation

Total Calculation: O(n) + O(mn) + O(n)

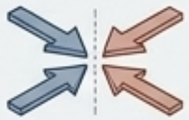
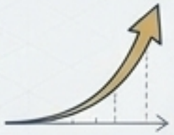
Reduced to: O(mn)



At algorithmic scale, the heaviest architectural component entirely dominates the execution time. Lower-order single loops disappear in the shadow of nested structures.

Gemini Notebook

The Big-O Diagnostic Matrix

Update Condition (Code)	Trajectory Pattern	Execution Count	Final Big-O
`i = i ± c`		Runs n times	`O(n)`
`Two Pointers (L → ← R)`		Covers n space once	`O(n)`
`i = i × c or i / c`		$k = \log_c n$	`O(log n)`
`i = i^c`		$k = \log_c \log_2 n$	`O(log log n)`
`Nested n loops`		`n × n × ...`	$O(n^{depth})$

Look for the update variable pattern. The math dictates the Big-O.

Gemini Notebook